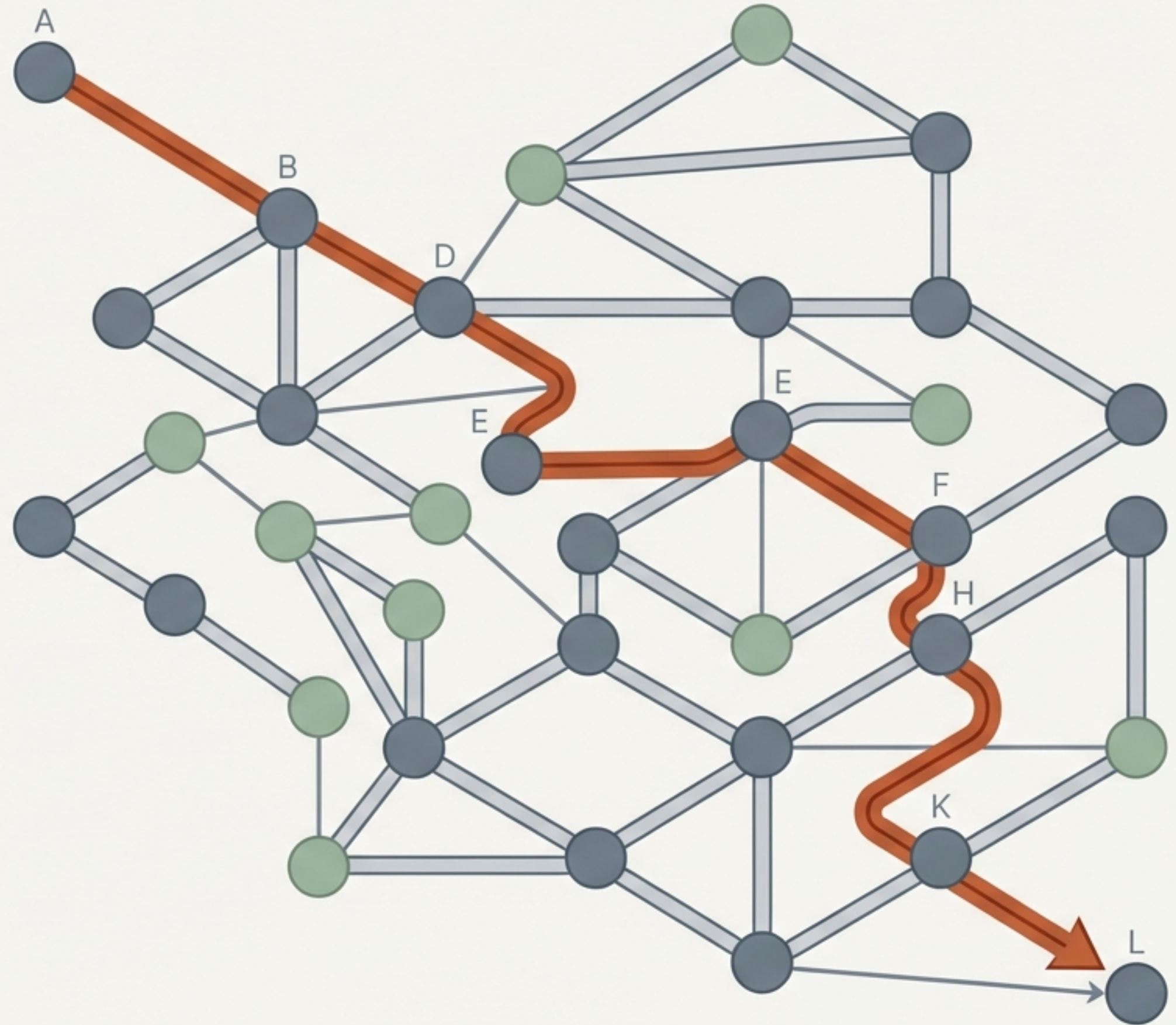


Graph Traversal: Decoding Depth First Search (DFS) in Python

DFS is a fundamental pathfinding algorithm that explores as deep as possible along each branch before backtracking, prioritizing depth over breadth.

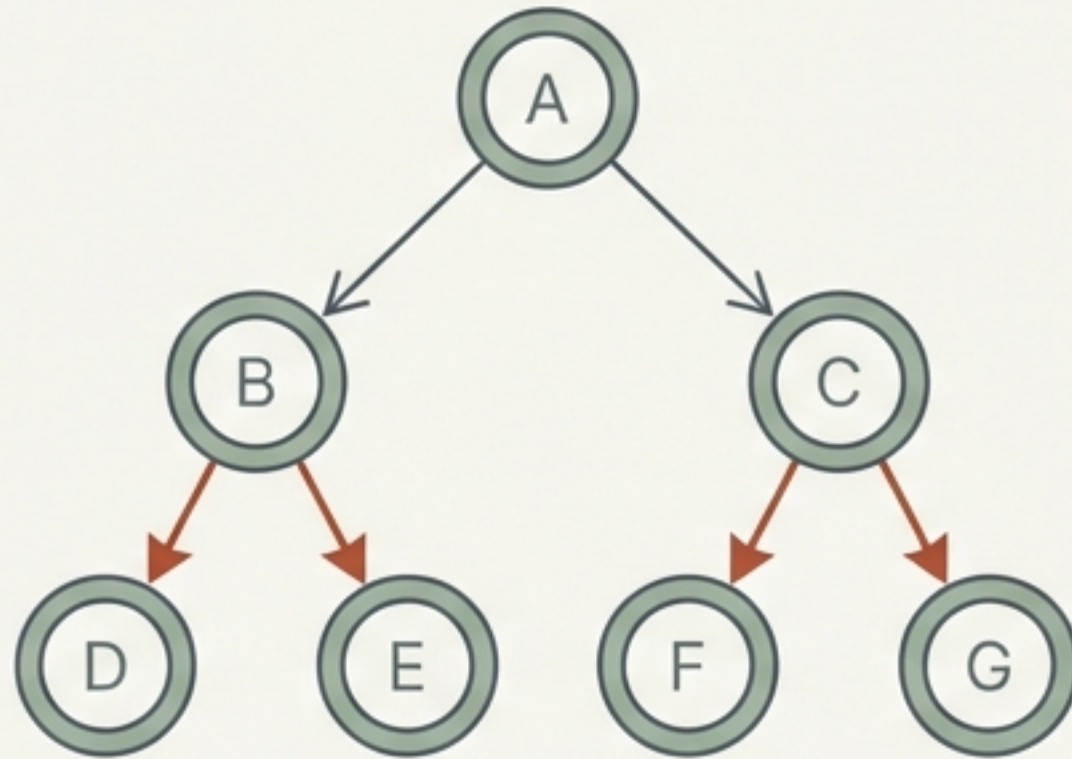
```
def dfs(graph, start, visited=None):  
    if visited is None:  
        visited = set()  
        visited.add(start)  
    print(start)  
    for next in graph[start] - visited:  
        dfs(graph, next, visited)
```

```
# Example usage:  
# graph = {'A': {'B', 'C'}, 'B': {'D', 'E'}, ...}  
# dfs(graph, 'A')
```



Every tree is a graph, but not every graph is a tree

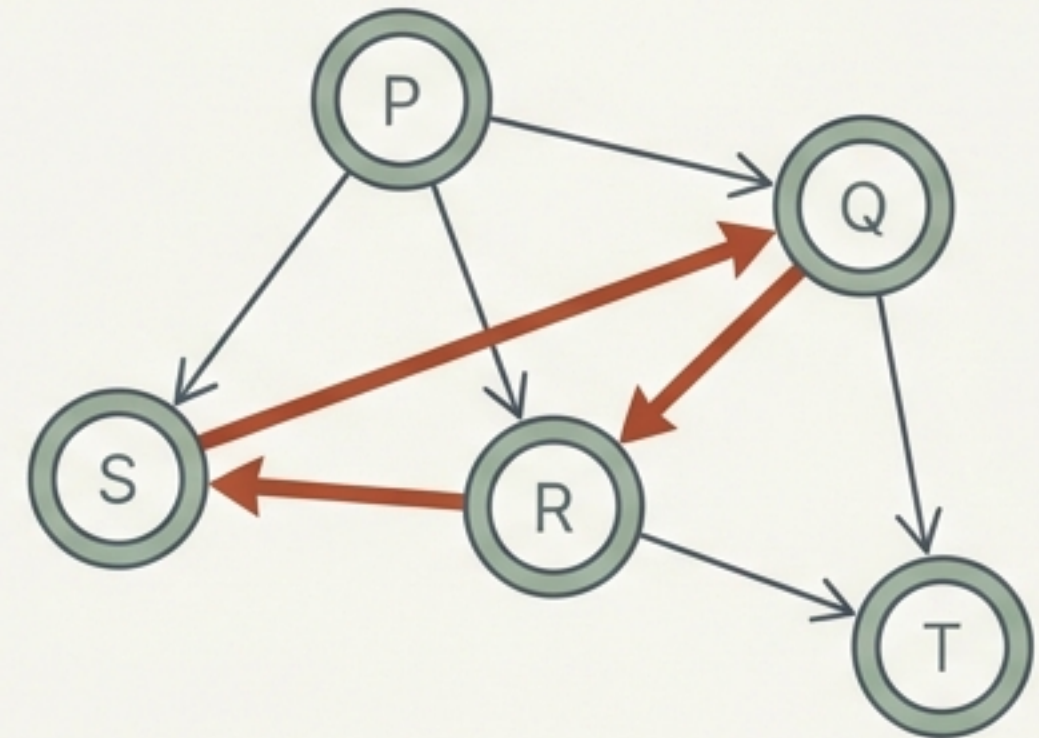
TREE STRUCTURE



Hierarchical. No repeating nodes.

```
// Key Properties in JetBrains Mono
struct Tree {
    bool is_connected; // True
    bool has_cycles;   // False
    // Fixed path from root to any node
};
```

GRAPH STRUCTURE

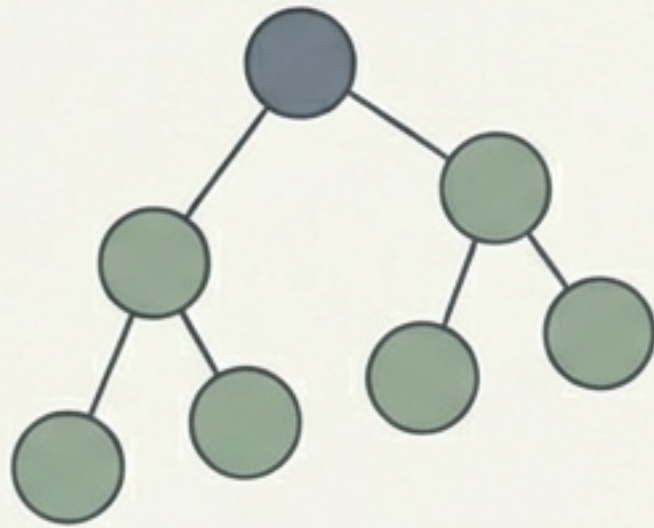


Cyclic. The game-changer is the ability to loop back to a previous vertex.

```
struct Graph {
    bool is_connected; // Can be true or false
    bool is_connected; // Can be true or false
    bool has_cycles;   // Can be true (e.g., Q -> R -> S -> Q)
    // Multiple paths possible between nodes
};
```

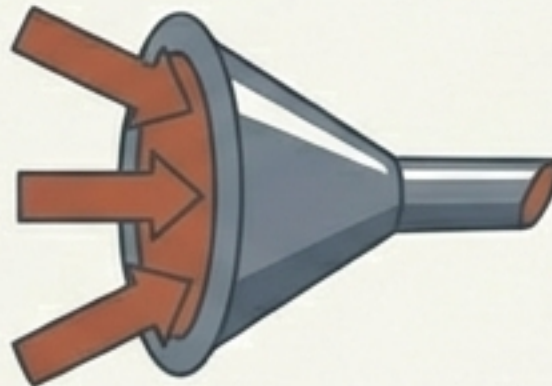
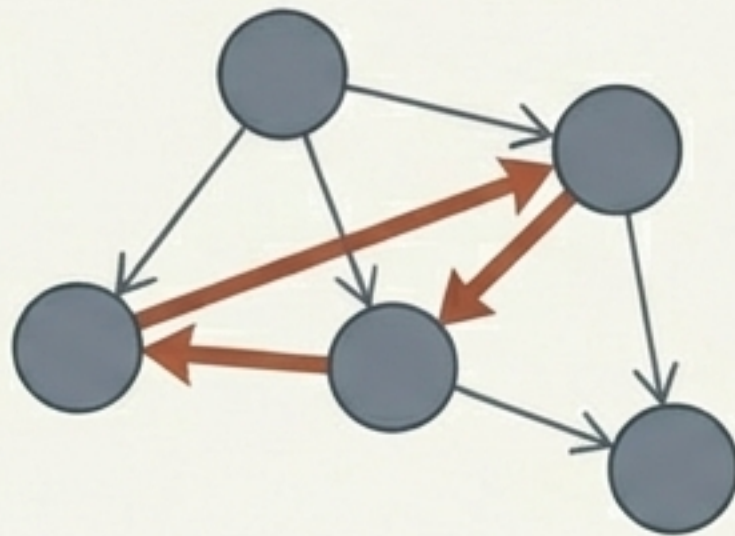
The “Multiple Answers” Secret

90% of students miss this crucial nuance.



Sequence
A

In trees, traversals yield exactly one unique output starting from a fixed root.



Sequence
A

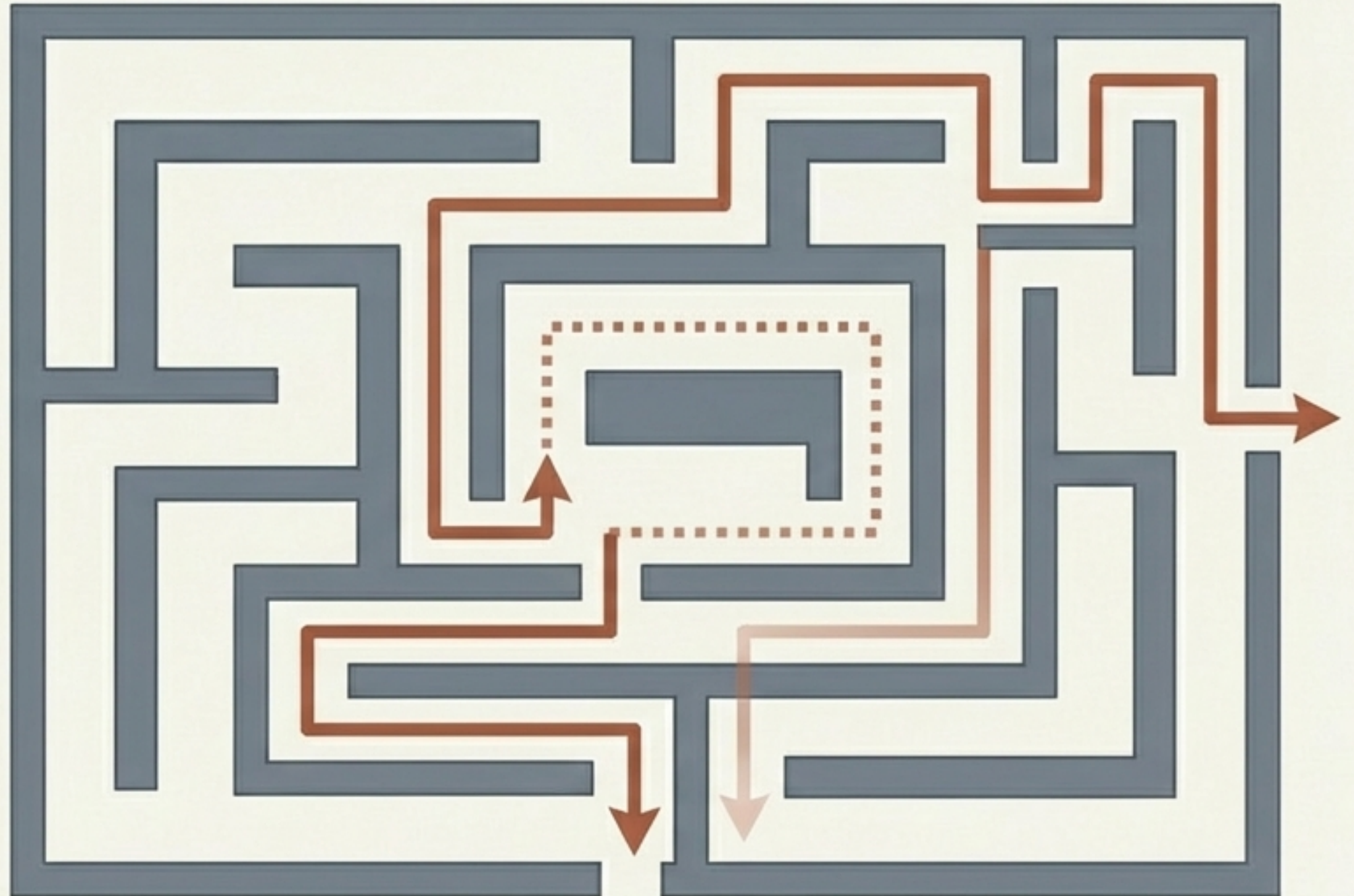
Sequence
C

In graphs, any node can be the starting point. Depending on the neighbor chosen, multiple different traversal paths are perfectly valid and correct.

The Maze Explorer's Intuition

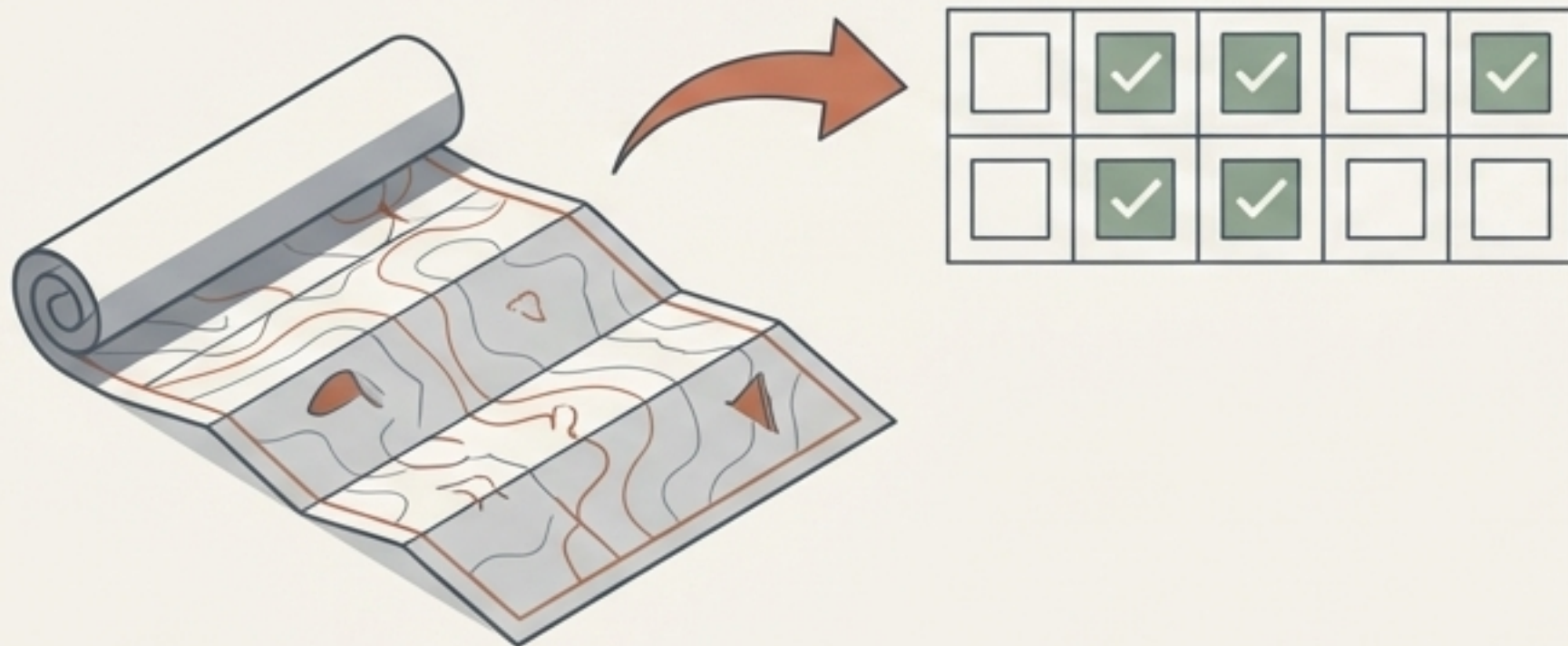
Treat DFS like escaping a maze.

- Rule 1: Go as deep as possible into the current path.
- Rule 2: Hit a dead end? Backtrack to the last intersection and try a new route.

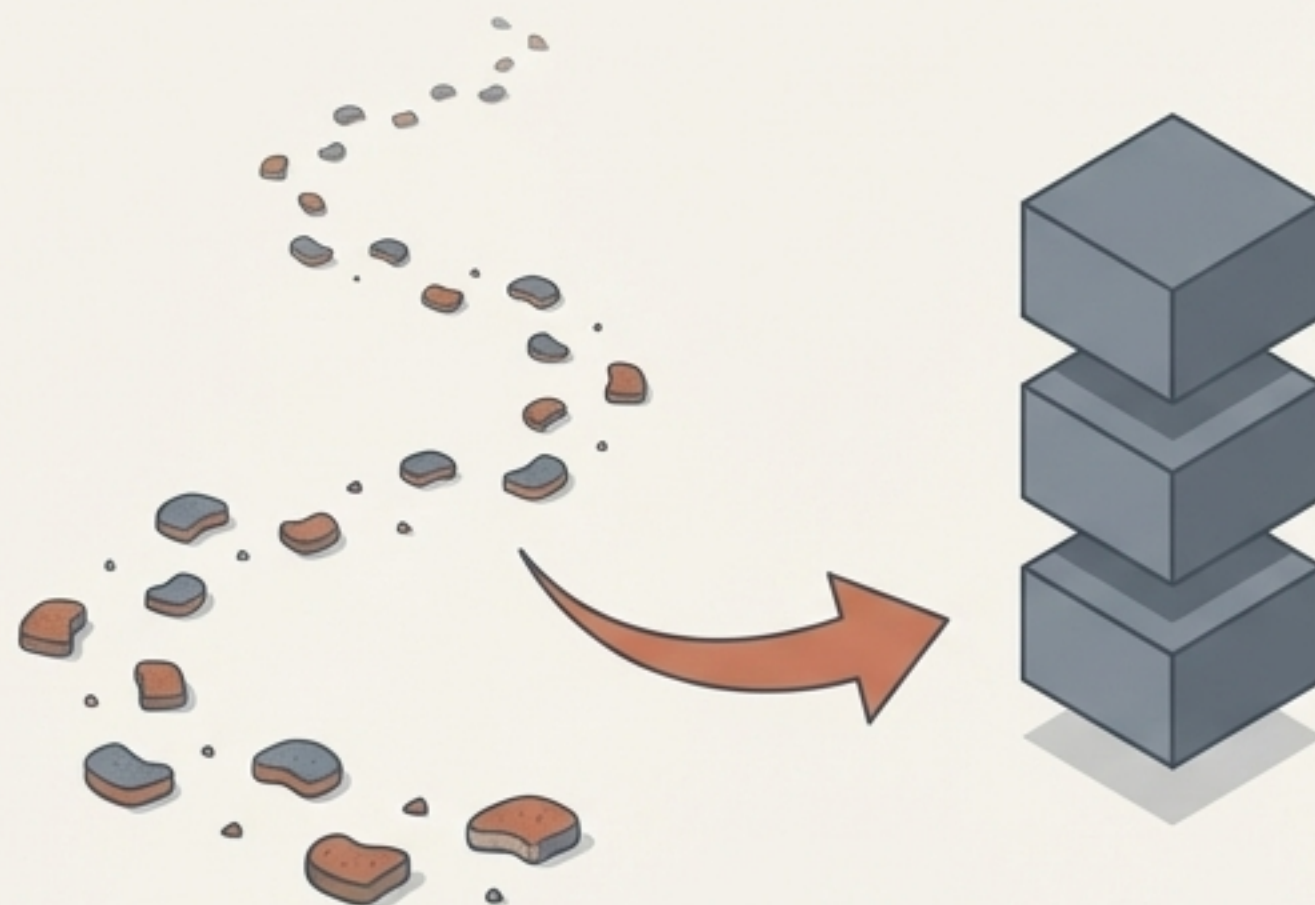


The DFS Survival Toolkit

Core tools for navigating complex structures.



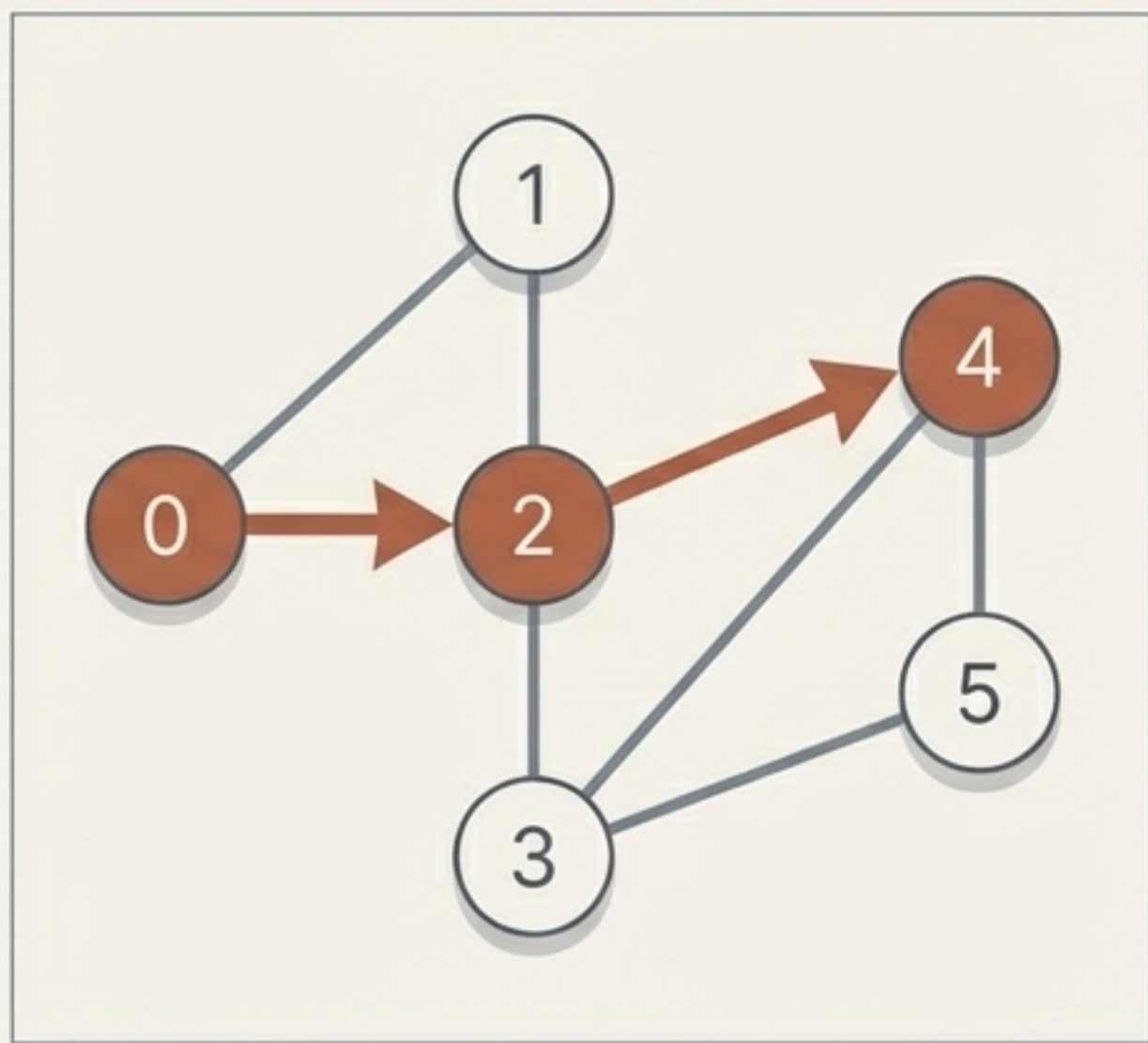
The Visited Array (The Map): Marks where we've been. Essential for preventing infinite loops in cyclic graphs.



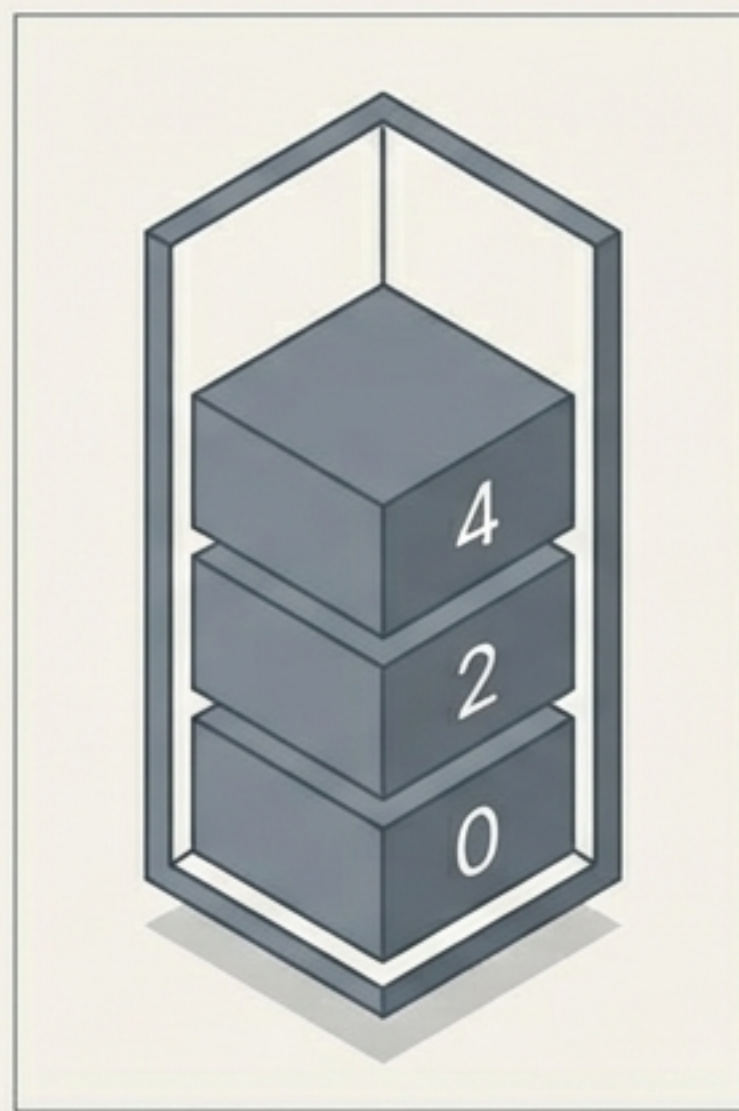
The Stack (The Breadcrumbs): Remembers where we came from. Essential for backtracking when we hit a dead end.

Visualizing the Run: Going Deep

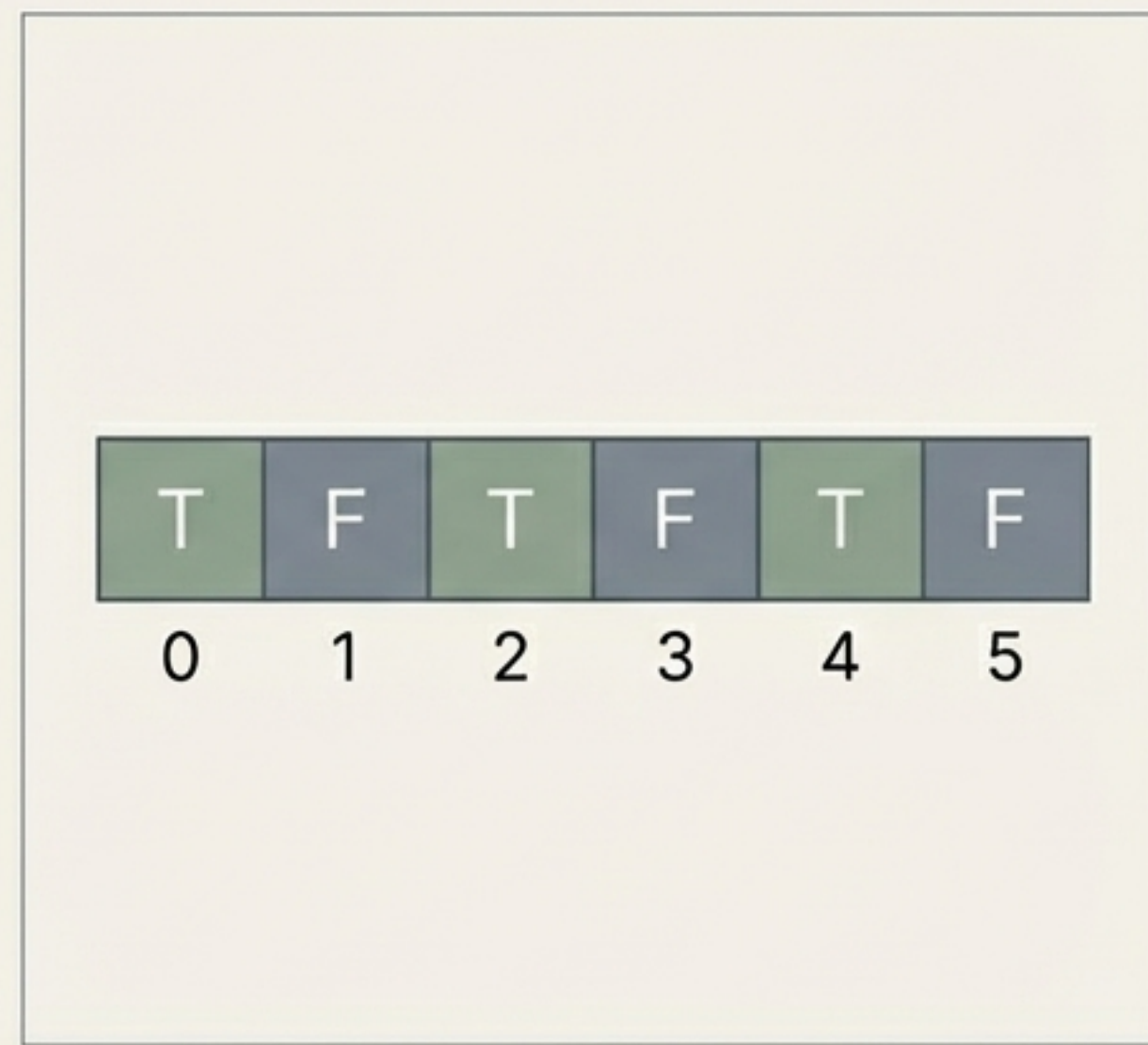
As long as there is an unvisited neighbor, we push to the stack and mark it True.



Panel 1: The Graph



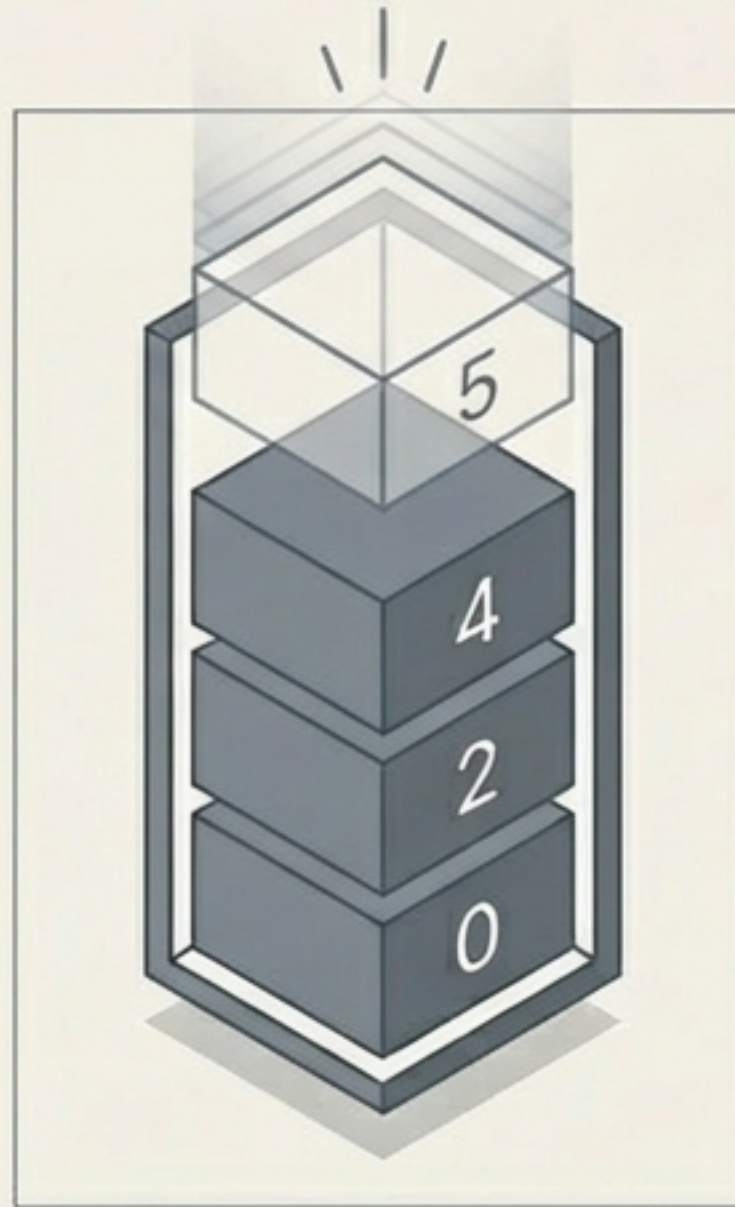
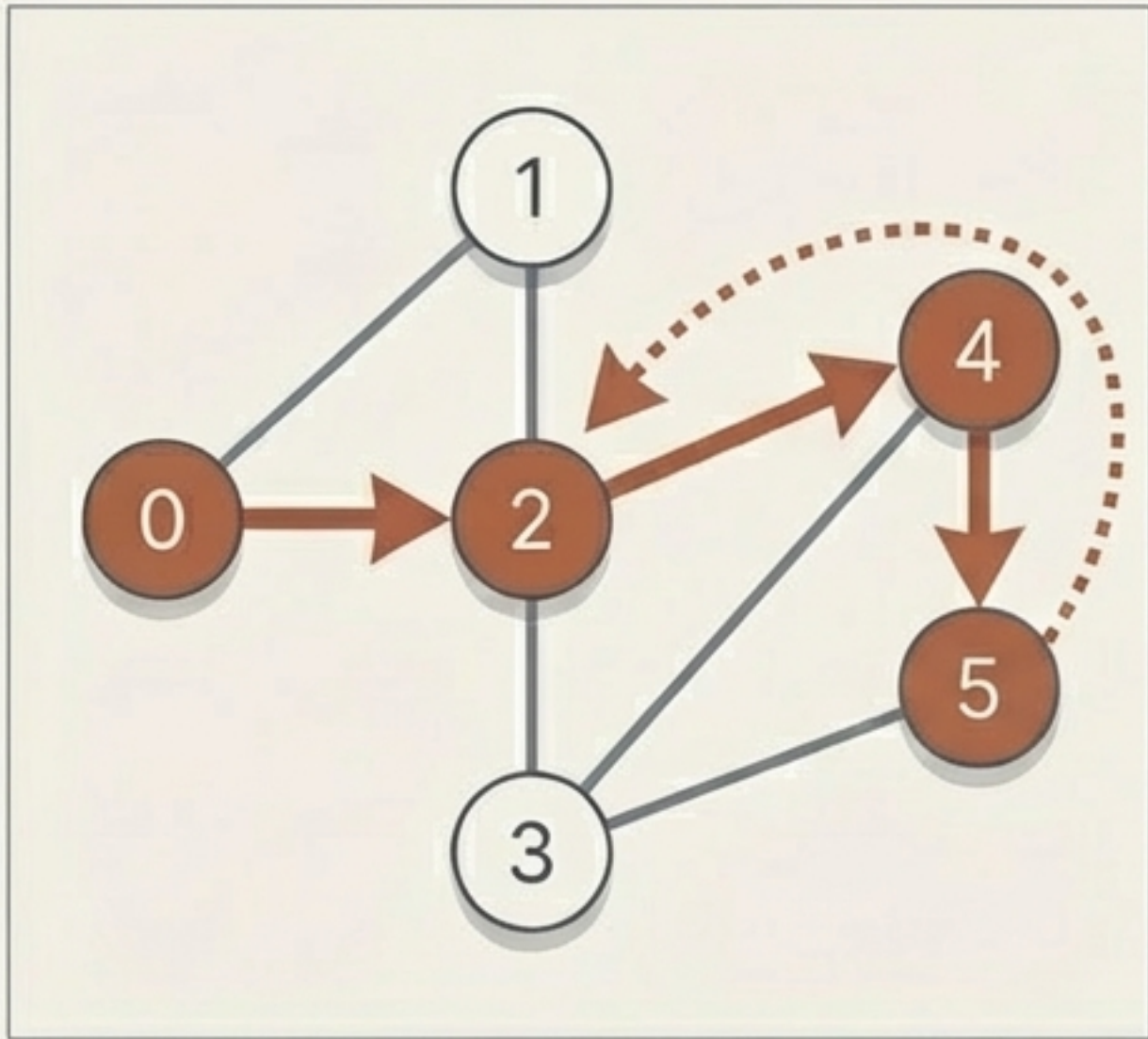
Panel 2: The Stack



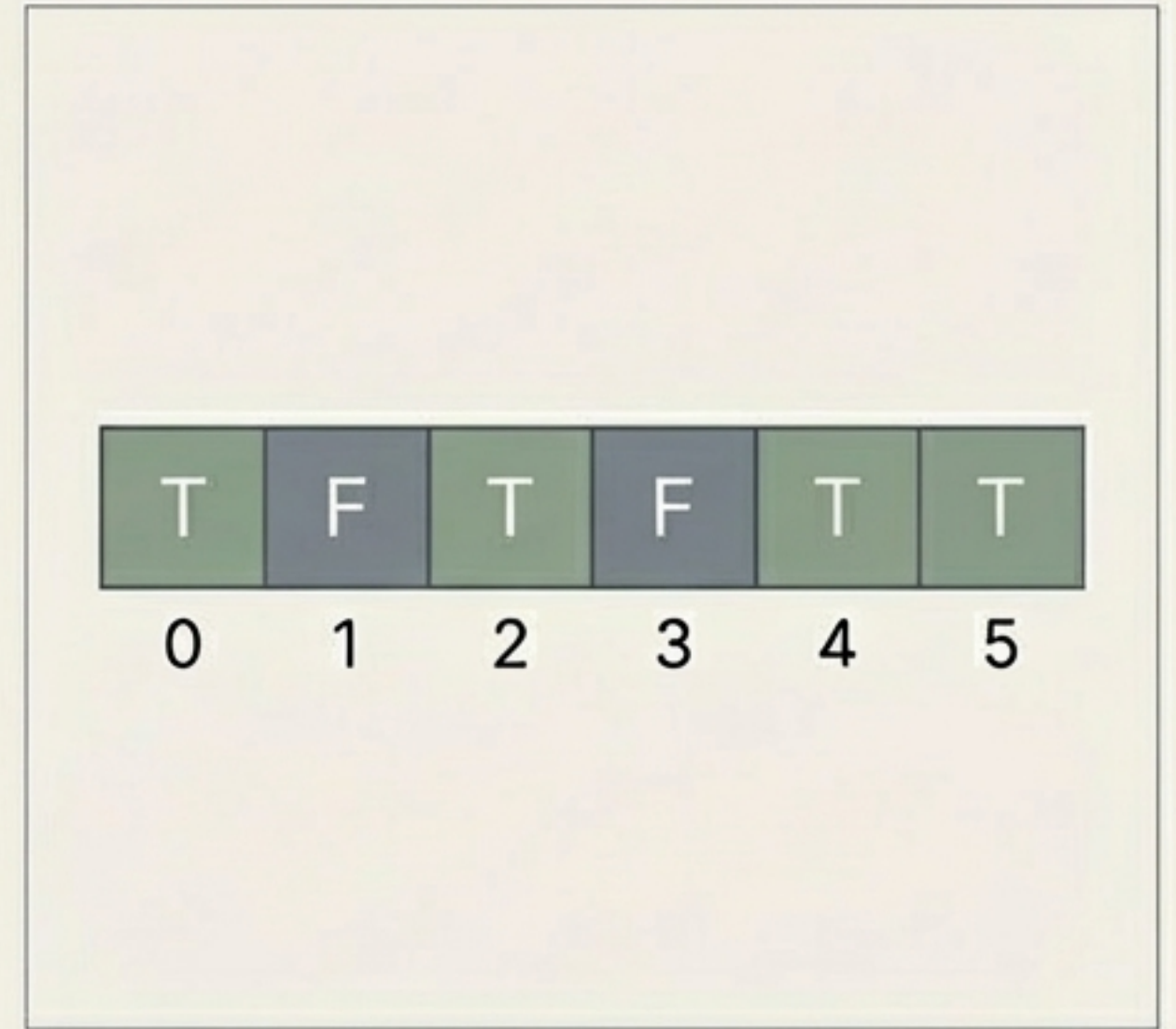
Panel 3: The Visited Array

Visualizing the Run: The Dead End

No unvisited neighbors? We Pop from the stack to backtrack and discover new paths.



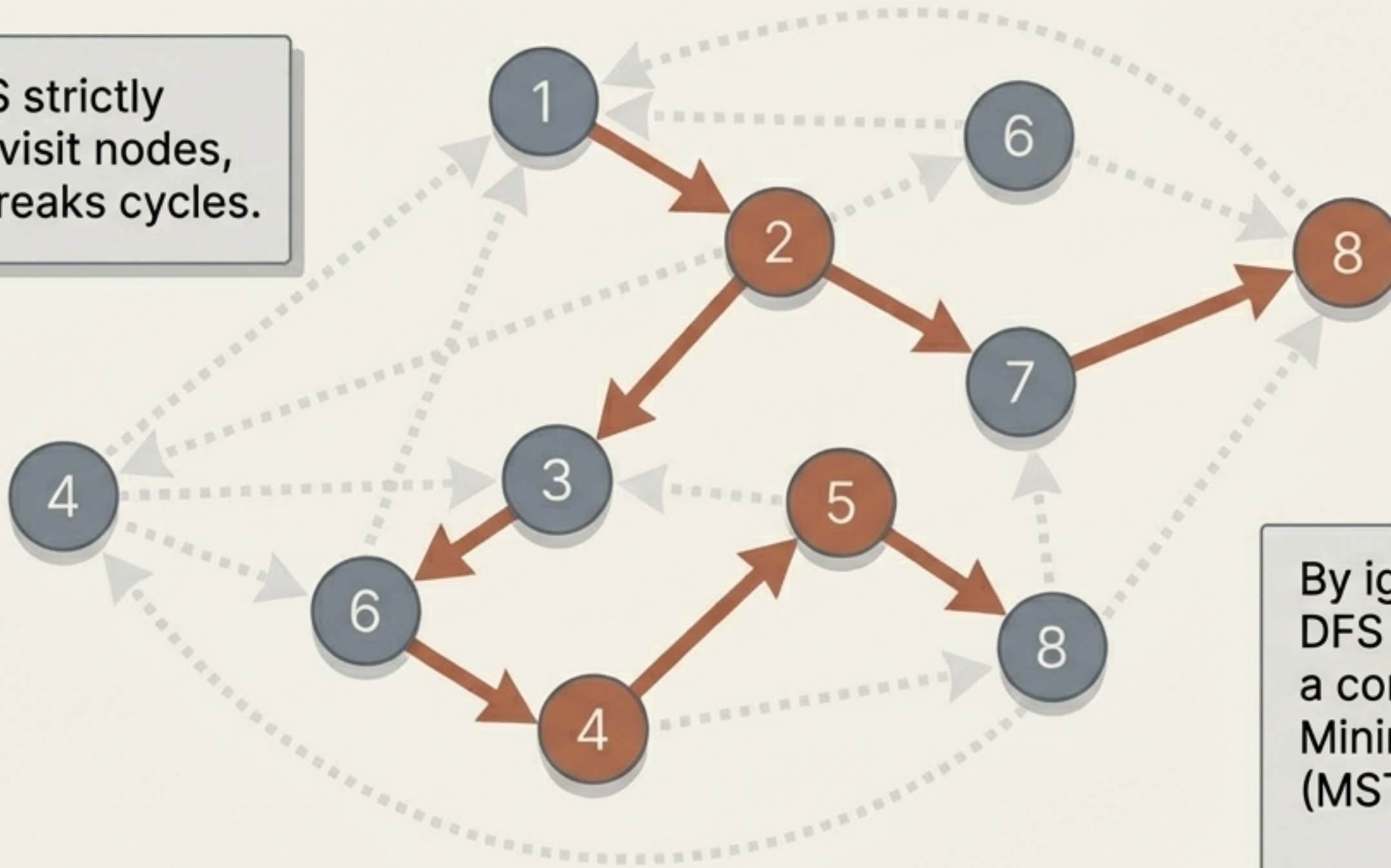
Panel 2: The Stack



Panel 3: The Visited Array

The Hidden Byproduct: Minimum Spanning Trees

Because DFS strictly refuses to revisit nodes, it naturally breaks cycles.

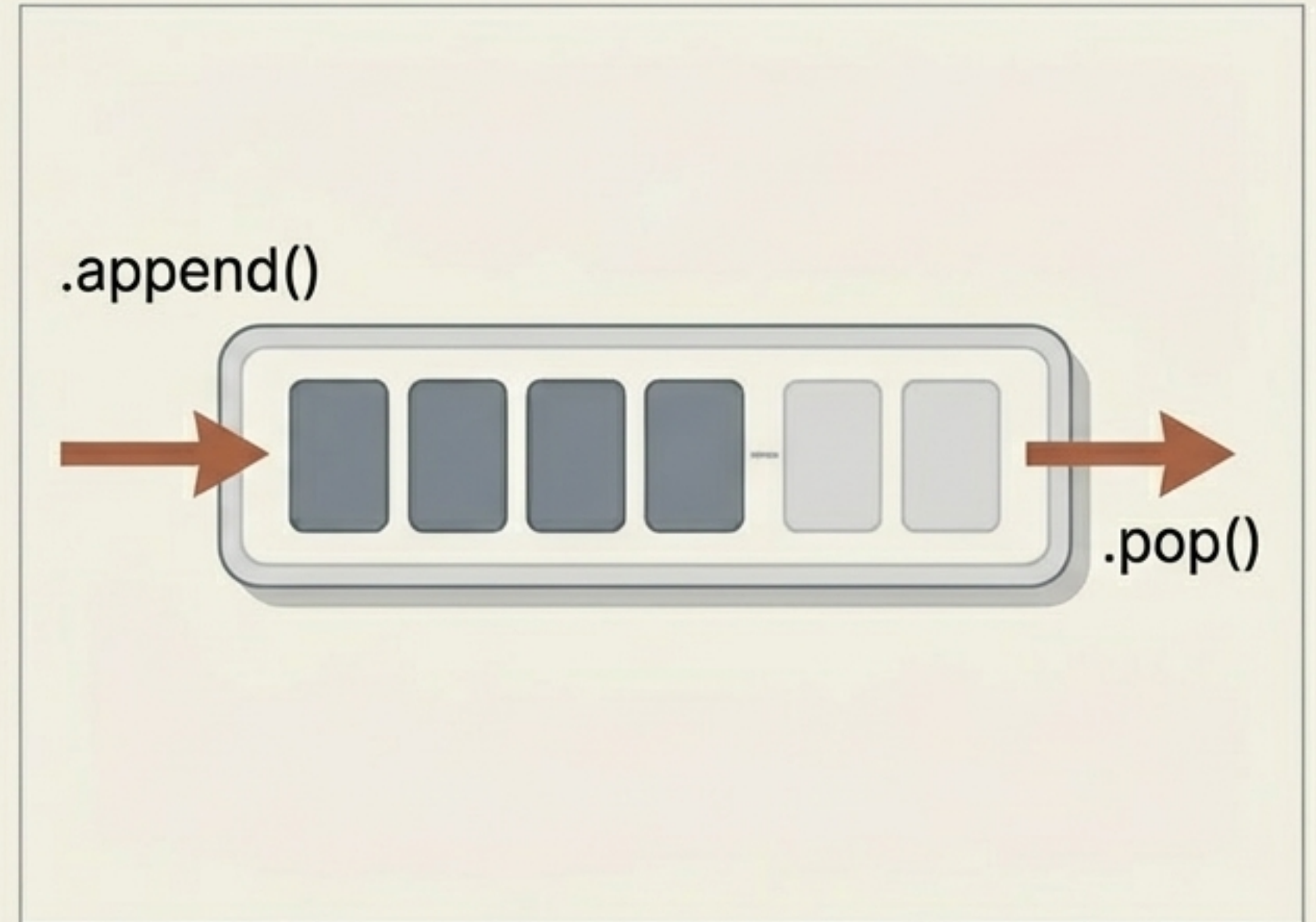


By ignoring cyclic edges, DFS inherently converts a complex Graph into a Minimum Spanning Tree (MST).

Translating Theory to Python

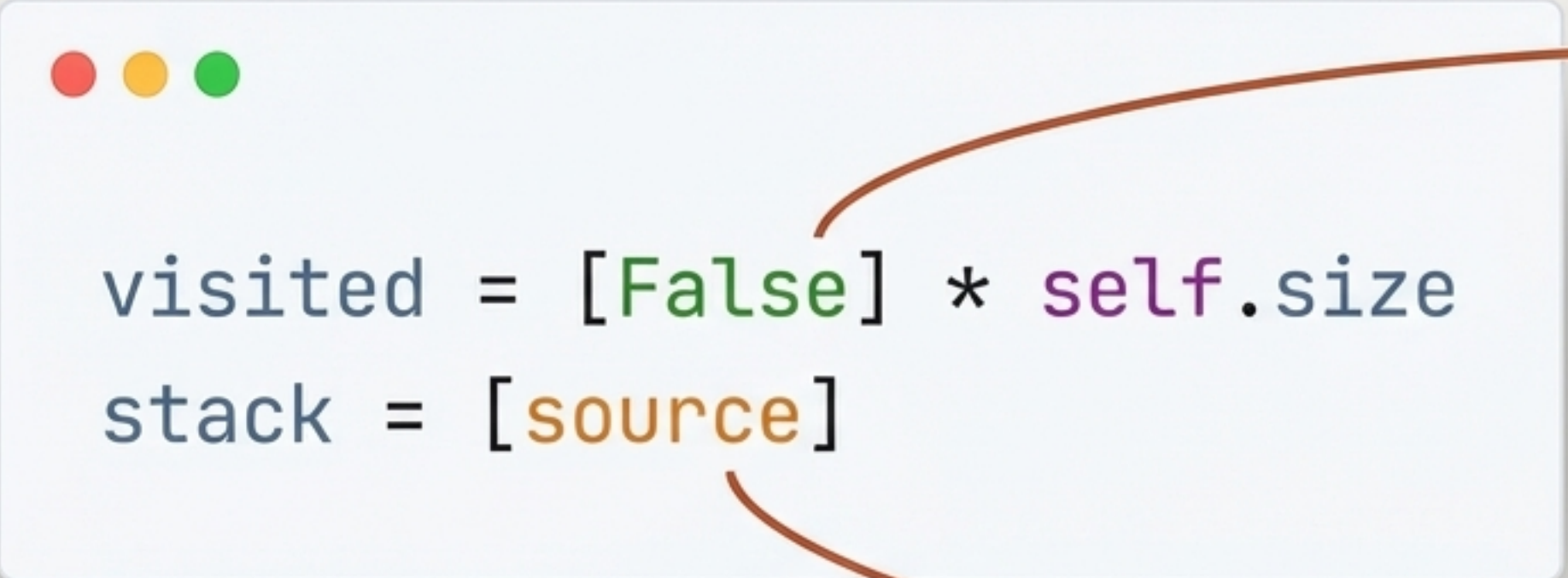
	0	1	2	3	4
0	0	1	1	0	0
1	0	0	0	0	0
2	0	0	0	0	1
3	0	0	0	0	0
4	0	0	0	0	0

Adjacency Matrix: We use a 2D grid to check connections (1 = edge exists, 0 = no edge).



Python Lists: Python doesn't need a custom stack class. A built-in List using `.append()` and `.pop()` acts as our perfect Stack mechanism.

Code Part 1: Environment Setup



```
visited = [False] * self.size
stack = [source]
```

Creates our “Map” array based on the total number of vertices.

Initializes our “Breadcrumb” stack with the chosen starting node.

Code Part 2: The Core Loop

```
1 while stack:  
2     v = stack.pop()  
3     if not visited[v]:  
4         print(v, end=" ")  
5         visited[v] = True
```

Runs until we've backtracked out of the entire maze.

Pulls the current location.

Marks the location on our map so we never process it twice.

Code Part 3: Finding Neighbors



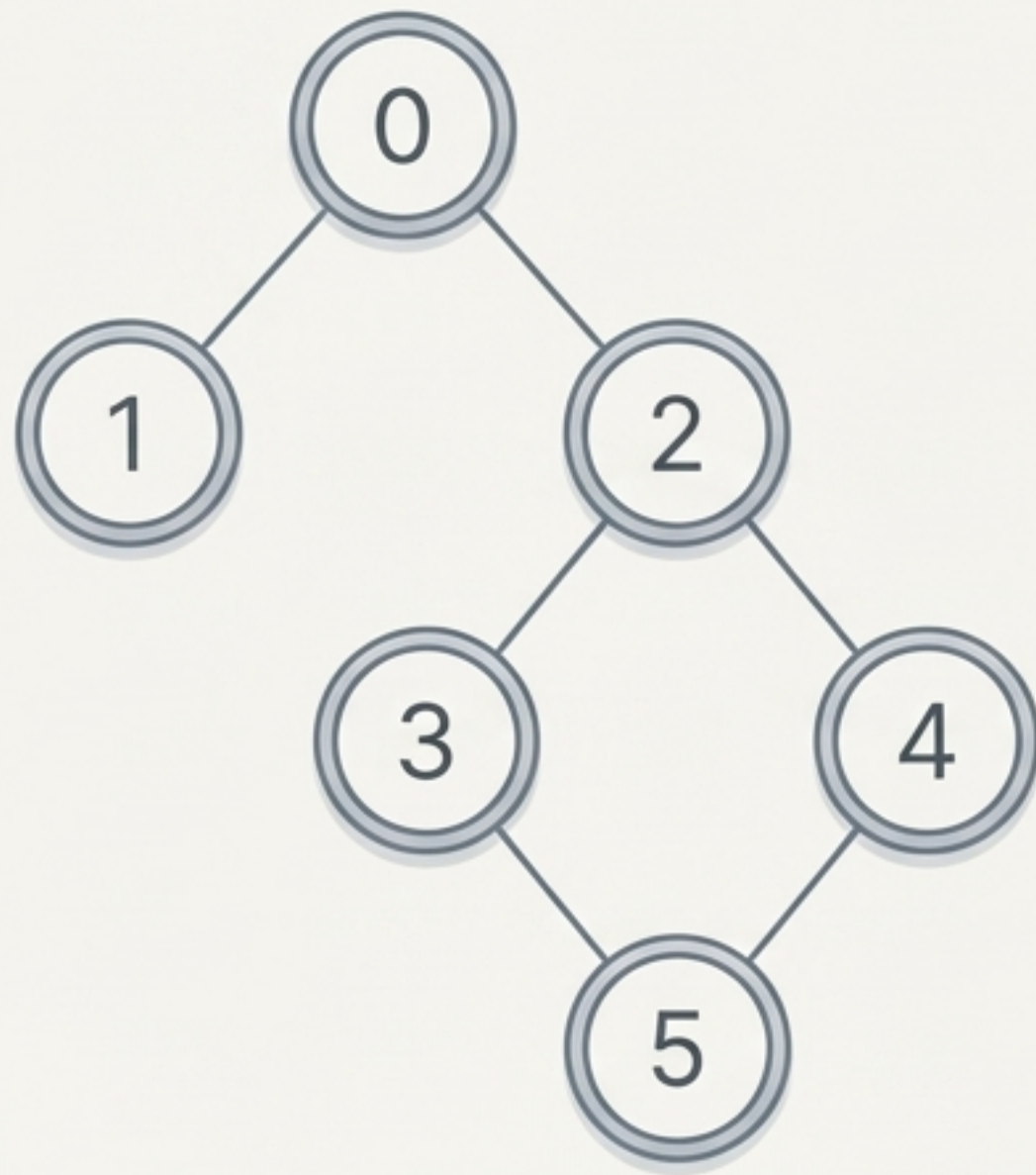
```
for i in range(self.size):  
    if self.matrix[v][i] == 1 and not visited[i]:  
        stack.append(i)
```

IF connected (matrix == 1) AND NOT visited (visited == False)
-> Append to Stack.

This pushes all viable paths onto the stack, ready to be explored in the next loop iteration.

Execution & Output Validation

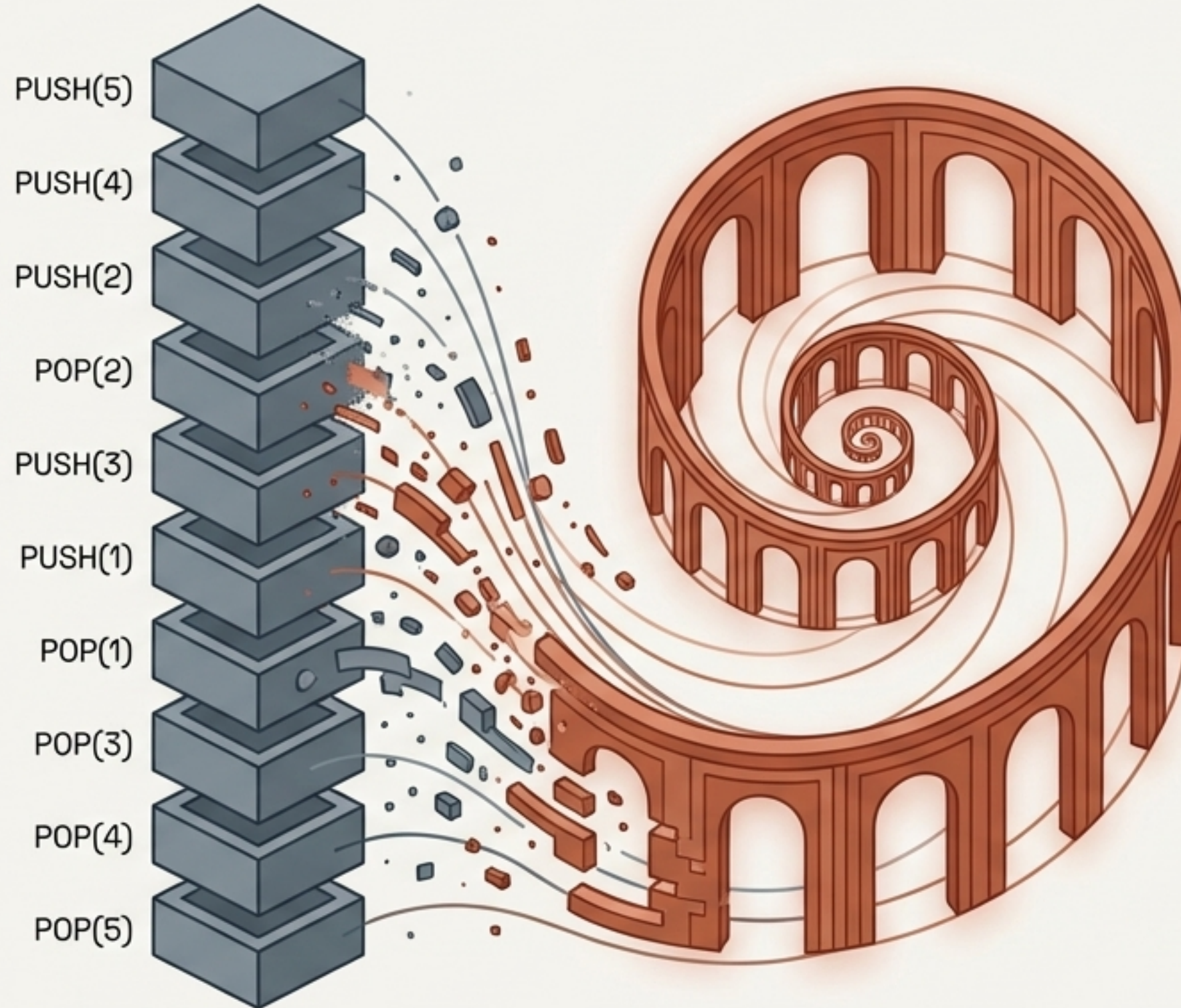
Starting at node 0, the Python stack perfectly replicates our logical maze dry run.



```
$ python dfs_graph.py  
0 2 4 5 3 1
```

The Next Challenge: Recursion

The **iterative stack** method builds deep intuition for how DFS tracks state.



Your next step?
Implement DFS using Recursion.

Ditch the manual list and let the system's own Call Stack handle the breadcrumbs.